

Trail — Interruptible Cursor Agent

Sep 28, 2026 · @Shresth Samyak

TL;DR

Trail is a desktop agent that lives with your cursor, remembers what you looked at, and handles interruptions in both directions. Hover three flight dates, ask "which one?", and it answers from what you saw. Hover a cheaper date while it is still talking, and it drops the stale answer and pivots mid-sentence.

Three mechanisms separate it from the cancel-and-restart agents most Theme 05 teams will submit:

- **Speculative forks.** While you are still talking, it pre-computes the 2–3 corrections you are most likely to make. When you make one, the answer is already done.
- **Two-way interruptibility.** You can interrupt the agent, and the agent decides when it may interrupt you: a security bug now, a style nit at your next save.
- **Transactional tools.** Every action is tagged reversible, compensable or irreversible. An interrupt mid-action rolls it back or holds it at a commit barrier.

The scored core is a harness-compliant interruptible runtime. The cursor overlay and the browser and VS Code agents are the demo layer built on top of it. Pitch line: a small local model for reflexes, a cloud model for thinking.

Problem and positioning

The Theme 05 baseline is already public, so Trail competes on what that baseline leaves out. That means interrupts that are not speech, interrupts the agent starts, and side effects caught mid-interrupt.

Public Theme 05 repos quote the brief as asking for the following. Confirm each against your own kit copy.

- Start retrieving before the utterance ends (full duplex).
- Handle and recover from real-time interruptions without losing logical depth.
- Handle goal changes without losing session context, and drive toward the end goal.
- A solid harness around the real-time agent.
- Session-scoped memory only.
- Multimodal input: text, image and voice, where voice may be simulated from transcripts.

The same repos list speech-synthesis quality, wake-word detection and UI polish as out of scope for grading. So the cursor overlay earns demo impact, not rubric points.

Capability	Public baseline (e.g. interject)	Trail
Barge-in	Turn is a cancellable task; checkpoint saved on cancel	Same, plus each interrupt is classified into one of 7 types before acting
Speculation	Retrieval starts on partial text	Full answers pre-computed for the 2–3 most likely corrections
Goal changes	Goal stack: continue, refine or park	Goal stack plus slot-level patching of the live plan
Tool safety	Irreversible tools cannot self-authorise	Saga model: rollback, compensation, or hold at a commit barrier
Input channels	Text, image, simulated voice	Adds a cursor attention stream (hover, dwell, app switch)
Who interrupts	User only	Both; an arbiter schedules the agent's own interruptions
Memory	Session store keyed by socket	Session-scoped attention trail shared across apps

The harness score is table stakes. The pitch lives in the last three rows, which none of the public entries reviewed here cover.

Product concept

Trail turns where you point into context, so questions need no setup and moving the pointer counts as an interruption.

- 1. Attention trail.** Every hover that dwells on readable content is logged with its text, app and timestamp. "Which one should I book?" is answered from that trail, not from a fresh search.
- 2. App-aware specialists.** The active app picks the agent: Chrome gets booking and shopping, VS Code gets the code mentor, a PDF viewer gets the research reader, Excel gets the sheet analyst. You can also pick one manually from the overlay.
- 3. Two-way interruptibility.** You interrupt Trail by voice, by keyboard, or by pointing somewhere else. Trail interrupts you only when its arbiter says the moment is right.

Story	You do	Trail does	Interrupt exercised
Flight pick	Hover Fri ₹6,400, Sat ₹5,000, Sun ₹11,000; ask "which should I book?"	"Saturday, ₹5,000, cheapest of the three you checked"	None; trail recall
Mid-answer data change	While it answers, hover Mon ₹4,500	Cuts itself off: "wait, Monday is cheaper at ₹4,500"	New evidence invalidates the answer
Pointer correction	Ask "what's this?" on a fare-rule icon, then point at baggage and say "no, this"	Drops the first explanation, parks it, explains baggage	Pointer shift + correction
Cross-app	Switch to your budget sheet; ask "can I afford the cheapest one?"	Compares ₹4,500 with the remaining-budget cell	None; shared trail across apps
Code mentor	Declare "building OAuth login", then type a bug	Waits for a typing pause; drops the warning if you fix it first	Agent-initiated, deferred
Security slip	Paste a secret into source	Interrupts at once	Agent-initiated, immediate
Test failure	A stack trace flashes; ask "why did that break?"	Answer already computed from the trace and the files you hovered	Speculative pre-diagnosis

Interruption model

Every interruption is classified before the agent acts on it, because a backchannel and a correction need opposite responses. Trail uses **duck-then-decide**: on any user activity it lowers or pauses output at once, classifies, then resumes, patches or stops.

User interrupts the agent

Type	Example	Detection signal	Handling
Backchannel	"mm-hm", "okay"	Short utterance, filler lexicon, no slot change	Resume output; nothing cancelled

Type	Example	Detection signal	Handling
Correction	"actually Saturday"	A known slot gets a new value	Stop output, patch that slot, serve the matching speculative fork if one exists
Addition	"and a window seat"	A new constraint, no conflict with existing slots	Keep the plan running; fold the constraint in at the next step
Cancel	"stop", "never mind"	Stop lexicon, or Esc	Hard stop, roll back reversible actions, keep the goal parked for "go back"
Topic switch	"what's the refund policy?"	New intent unrelated to the active goal	Checkpoint and park the goal; offer the way back once the detour ends
Clarification	"what does that mean?"	Question about the agent's own last output	Pause, answer inline, resume from the checkpoint
Pointer shift	"no, this" while pointing elsewhere	Hover target changes during a deictic question	Rebind "this" to the new target, re-answer, park the old one

Classification runs in two stages. Rules on the transcript, slot diff and pointer events decide most cases. The local model settles only the ambiguous remainder.

Agent interrupts the user

The **Interrupt Arbiter** owns every unsolicited message. It scores severity, waits for the right moment, and re-checks validity just before speaking.

Tier	Examples	When it may speak
Critical	Secret in source, an irreversible action about to fire	Immediately
High	Logic bug in the line just written; new evidence contradicts the live answer	Next micro-pause: typing idle or the user's sentence ends
Normal	A better approach, drift from the declared goal	Natural boundary: save, commit, test run, app switch

Tier	Examples	When it may speak
Low	Style, naming	Batched digest, only when asked

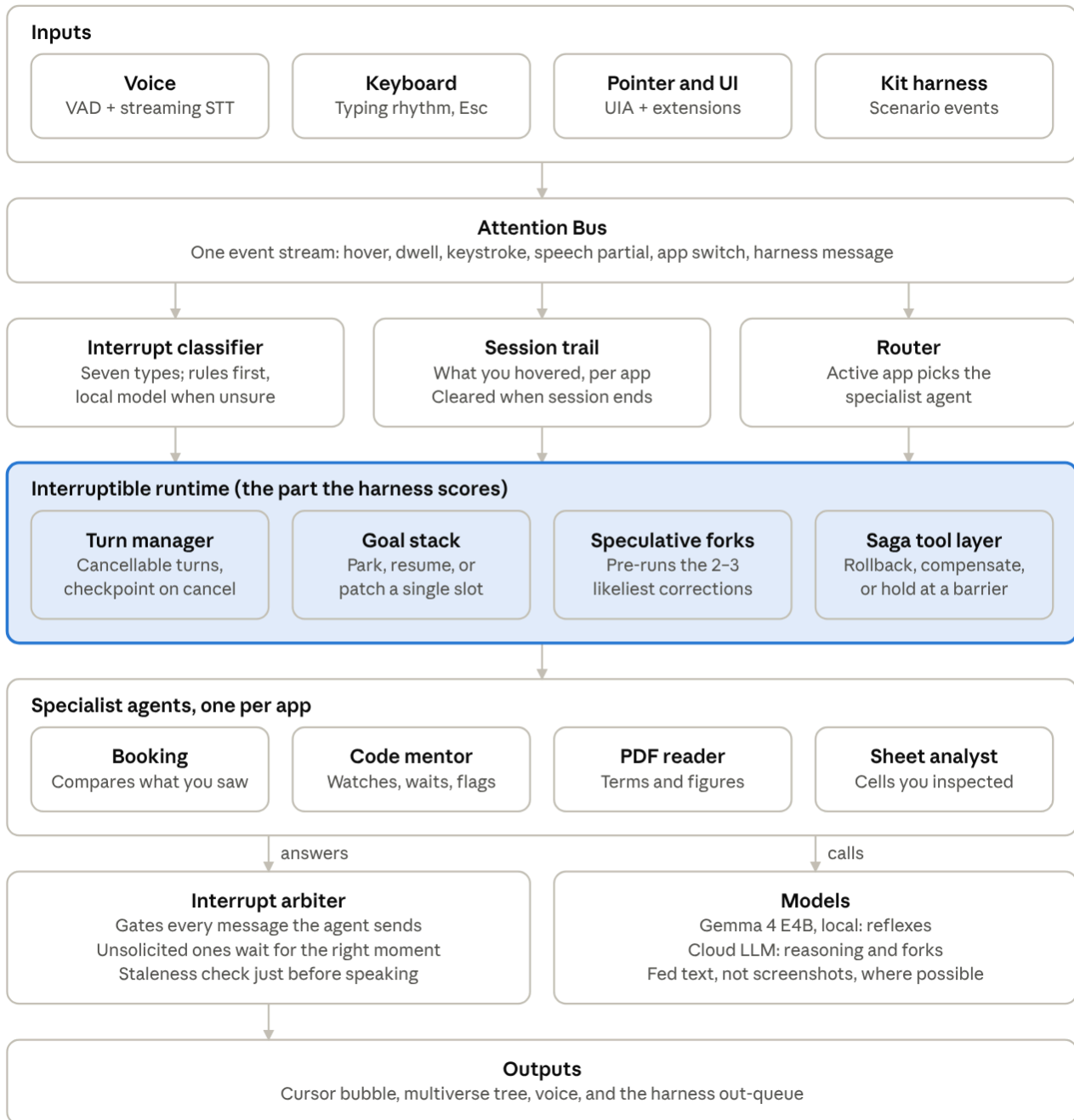
Three rules keep the arbiter from being annoying:

- **Staleness check.** Before speaking, it re-reads the code or page version. If the user already fixed the issue, the message is dropped silently.
- **Never talk over speech.** While the user is speaking, only Critical may break in.
- **Interruption budget.** Unsolicited messages are capped per 10 minutes. "Not now" raises the tier threshold for a while.

System architecture

Every input, including the kit harness, is normalised into one event stream. That way the scored path and the demo path run the same runtime code.

All inputs share one event stream; one runtime decides what to say and when



Trail architecture · 7 layers, one event stream

Read it top down: events feed classification, memory and routing; the runtime turns them into cancellable work; the arbiter gates everything that goes out. The classifier and router also call the local model.

Two run modes share this code:

- **Harness mode.** A thin adapter turns kit messages into bus events and arbiter output into out-queue messages. Perception and the overlay stay off.

- **Desktop mode.** Perception sources feed the bus. Output goes to the cursor bubble, the multiverse tree and voice.

Interruptible runtime

The runtime is one asyncio loop in which every turn, fork and tool call is a cancellable task stamped with a state version. Stale work can never reach the user. This is the part the harness scores, so it is built and tested first.

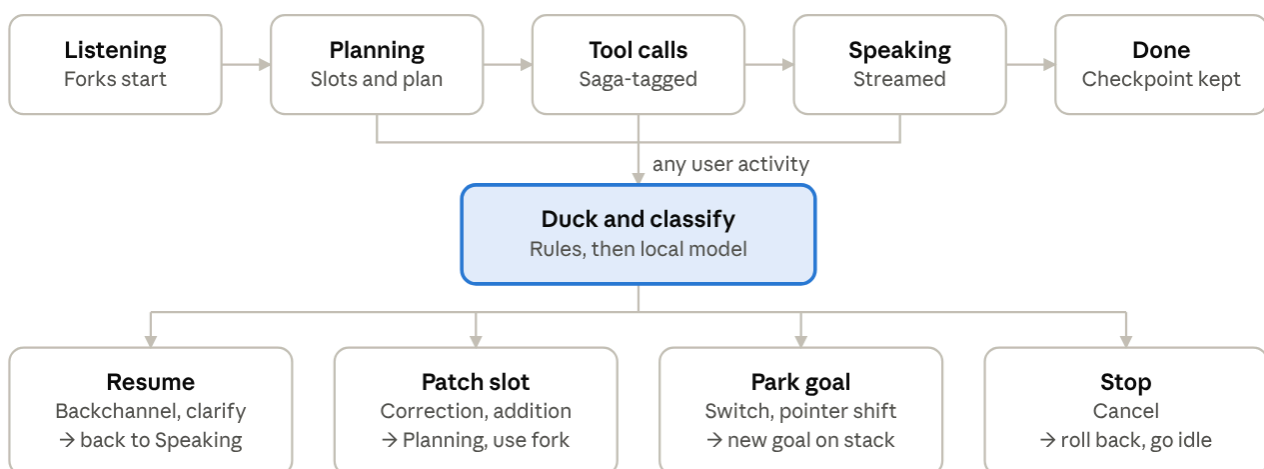
Harness fit

Public Theme 05 repos describe the kit roughly as follows. Map every message type in your kit's protocol reference to a bus event before writing any logic.

- The agent is a `ParticipantAgent(in_queue, out_queue)` class. `run()` takes no arguments, and an optional `async def setup()` runs first.
- Scenario files such as `scenarios/pub_02_text_interrupt.json` run locally through `run_local.py`, which supports `--all` and `--time-scale`.
- `eval_submission.py` dry-runs the official evaluator. The kit's minimal reference agent scores about 52/100 on the public set.
- API keys are registered on the event portal and exposed as `SECRET_*` environment variables.

Turn lifecycle

Output ducks on any user activity; the classifier then picks one of four outcomes



Turn lifecycle · 5 states, 4 interrupt outcomes

Output is ducked the instant the user does anything; the outcome is chosen only after classification, so a backchannel costs nothing.

Mechanisms

- **Versioned state, single writer.** One `SessionState` carries a version that increments on every change. Only the runtime loop mutates it; tasks return proposals, and any proposal stamped with an old version is dropped.
- **Cancellation discipline.** Each turn is one `asyncio.Task`. On cancel it writes its checkpoint synchronously and re-raises; nothing awaits I/O while unwinding.
- **Checkpoints.** A checkpoint holds the partial answer, the evidence gathered, the plan position and any pending tool calls. "Go on" resumes from it without redoing retrieval.
- **Goal stack with slot dependencies.** Each goal holds an intent, slots, a checkpoint and a status: active, parked, done or abandoned. Each plan step lists the slots it reads, so a correction re-runs only the steps that depend on the changed slot.
- **Speculative forks.** When a slot gets a value with cheap alternatives (date ± 1 day, cabin, quantity), up to 3 forks run with those values. Forks use idle time only, carry a token and tool budget, and die on any version change they do not match.
- **Saga tool layer.** Every tool in the manifest is tagged reversible (search, hold), compensable (book, then cancel) or irreversible (pay, send). An interrupt undoes or compensates the first two. Irreversible calls wait at a commit barrier for an explicit confirmation given after the last interrupt.
- **Idempotent calls.** Every tool call carries a `call_id`, so a retried or duplicated call never writes twice.
- **Fillers.** A short acknowledgement covers slow work so there is no dead air. Fillers never enter the transcript or the model context.

Attention Bus and perception

Perception reads UI text through accessibility and extension APIs first and uses vision only for pixels. Most hovers therefore cost no model call at all.

Source	Covers	How it reads the target
Browser extension (Chrome, Manifest V3)	Booking and shopping sites, web apps	Content script gets the element under the pointer and its nearest labelled container, e.g. a fare cell with its date and price
VS Code extension	Code mentor	Hover position, selection, document text, diagnostics, save events, terminal output via shell integration

Source	Covers	How it reads the target
Windows UI Automation (pywinauto)	Excel, PDF readers, native apps	Element under the pointer, its name and value, plus its row and column headers
Vision fallback (Gemma 4 E4B)	Canvas, images, scanned PDFs	Screen crop around the pointer, sent only on dwell, never per frame
Microphone	Voice	Silero VAD for barge-in, streaming STT for partial transcripts
Keyboard hook	Typing rhythm, Esc	Timing only outside VS Code; key contents are never recorded

Event model

Every source emits the same envelope, so the runtime never knows which source produced an event.

Event	Emitted when	Main consumers
<code>dwell</code>	The pointer rests on one target past the dwell threshold	Trail store, pointer-shift detection
<code>hover</code>	The pointer enters a new target (debounced)	Speculative prefetch
<code>select</code>	Text or cells are selected	Trail store, deictic "this"
<code>app_switch</code>	The foreground app changes	Router, arbiter (natural boundary)
<code>typing</code>	A typing burst starts or ends	Arbiter (flow state)
<code>save</code> / <code>test_run</code>	A file is saved or tests run	Arbiter (natural boundary), code mentor
<code>vad_start</code> / <code>vad_end</code>	Speech starts or stops	Duck-then-decide
<code>speech_partial</code> / <code>speech_final</code>	STT emits text	Classifier, speculative forks
<code>harness_msg</code>	The kit sends a message	Harness adapter

```
{
  "type": "dwell",
  "app": "chrome",
  "ts": 1727523456.12,
  "source": "extension",
  "target": {
    "role": "cell",
    "text": "Sat · ₹5,000",
    "context": "Chandigarh → Goa",
    "bbox": [812, 440, 960, 470]
  }
}
```

Dwell detection

- A dwell fires when the pointer stays on the same element for about 350 ms. That is a starting value to tune.
- Fast pointer transit is ignored, so sweeping across a page does not flood the trail.
- The extension pairs values with their labels using DOM proximity: the nearest container holding both a date-like and a price-like string.

Specialist agents

Build two specialists properly, booking as the hero and the code mentor as the second act, and pitch the rest as the same core applied to more apps.

Agent	Priority	Activates on	Reads	Signature interrupt behaviour
Booking	Hero demo	Browser on travel or shopping sites	Price and date pairs from the trail, the route on the page	Cuts off its own answer when you hover cheaper data
Code mentor	Second act	VS Code in the foreground	Diagnostics, diffs, saves, test runs, terminal output	Waits for your pause; drops warnings you already fixed
PDF reader	Stretch	PDF viewer	Hovered terms, equations, figure captions	"No, this" re-targets the explanation mid-sentence
Sheet analyst	Stretch	Excel	Hovered cells with their row and column headers	Answers over the cells you inspected, not the whole sheet
Email and calendar	Pitch only	Mail client	Hovered dates and names	"Block that" drafts an event and waits for a yes

Booking agent (hero)

- **Grounded answers.** It answers only from trail entries and names them: "of the three dates you checked, Saturday is cheapest".
- **Kit corpus.** Public repos say the kit ships a travel corpus covering fares, hotels, policy and support. Use it for fare rules and baggage questions, so the hero demo also exercises the scored domain.
- **Speculation.** A dwell on one date prefetches neighbouring dates and the fare rules for that date.
- **Saga-tagged tools.** Search and hold are reversible, booking is compensable, payment is irreversible and waits at the commit barrier.

Code mentor (second act)

It is a reviewer watching over your shoulder, not an autocomplete, so it never competes with Copilot.

- **Declared intent.** You say "I'm building OAuth login"; that becomes the session goal. Code that drifts from it, such as hand-rolled JWT parsing, is flagged as drift, not as a syntax error.
- **Three check layers.** Linter diagnostics cost nothing and run first. The local model screens each changed hunk. The cloud model does deeper review, but only at natural boundaries.
- **Secret detection.** Pattern checks for API keys and tokens are the only findings that interrupt instantly.
- **Staleness.** Every finding stores the document version and a hash of its line. It is re-verified just before delivery and dropped if the line changed.
- **Terminal pre-diagnosis.** A stack trace starts a speculative diagnosis over the files you recently hovered, before you ask.
- **Teach mode vs fix mode.** Teach mode asks a Socratic question, such as "what happens if `user` is None here?". Fix mode proposes the change.

Cross-app memory

The trail is an in-memory store of what you looked at, wiped when the session ends. That keeps it inside the brief's session-only memory rule while still joining facts across apps.

What a trail entry holds

- Timestamp, app, source, the target's text and its context (for example the route above a fare grid).

- Typed entities parsed from the text: prices with currency, dates, times, flight numbers, code symbols, cell references.
- Dwell time, used as a proxy for attention.

Parsing is rule-based (currency and date parsers, symbol names from VS Code, cell references from UI Automation). The local model is called only when the rules fail.

Resolving what the user means

The user says	Resolves to
"this"	The current dwell target
"that", "the other one"	The previous dwell target
"the cheapest", "the fastest"	Min or max over entities of that type within the active goal
"what I saw on MakeMyTrip"	Entries filtered by app or site
"can I afford it?" in Excel	A join: the price entity from the browser trail against the hovered budget cell

Ranking

When several entries match, each is scored by match quality, recency and attention:

$$\text{score}(e) = \text{match}(e, q) \cdot e^{-\Delta t / \tau} \cdot \left(1 + \ln\left(1 + d / d_0\right)\right)$$

Here Δt is the entry's age, d its dwell time and d_0 the dwell threshold. Start with τ at 10 minutes and tune it.

Password fields, payment fields and anything the extension marks sensitive never enter the store.

Models, speech and latency

A small local model handles reflexes and a cloud model handles reasoning. The harness path must work with the cloud model alone, because the kit's evaluator almost certainly cannot reach a GPU on your laptop.

Model split

Role	Model	Runs on	Notes
Scored core: planning, answers, forks	Fast cloud LLM allowed by the kit	Cloud	Pick for time-to-first-token, not raw benchmark scores
Ambiguous interrupts, routing, hunk screening, vision fallback	Gemma 4 E4B , 4-bit	RTX 4060 laptop (8 GB VRAM)	Desktop mode only; harness mode falls back to rules plus the cloud model
Code review at boundaries, cross-app answers	Same cloud LLM	Cloud	Called only at natural boundaries
Barge-in detection	Silero VAD	CPU	Fires within a speech frame; no model call
Speech to text	faster-whisper (small or distil), or a cloud streaming STT	GPU, or CPU int8 if VRAM is tight	Must stream partials; utterance-level audio models do not
Text to speech	Any fast local TTS	CPU	Synthesis quality is outside the grading scope

Why E4B and not the 12B: the edge models cover text, audio, vision and video at about 4.5B effective parameters (7.9B including embeddings) ([NVIDIA](#)). Google sizes the 12B for laptops with 16 GB of VRAM ([Google](#)). On 8 GB it would fight STT, VS Code and Chrome for memory.

Latency budget

These are targets to measure against, not measured results.

Path	Target	What it takes
Speech onset to output ducked	Under 150 ms	VAD frame plus duck; no model
Interrupt classified by rules	Under 20 ms	Lexicon, slot diff, pointer events

Path	Target	What it takes
Interrupt classified by local model (ambiguous cases only)	Under 300 ms	Short E4B prompt, constrained output
Correction to first token, fork hit	Under 300 ms	Serve the pre-computed answer
Correction to first token, fork miss	Under 1.2 s	Cloud time-to-first-token plus one tool call
Hover to trail entry	Under 50 ms	Extension or UI Automation; no model

The biggest single latency win is feeding models text from the DOM and UI Automation instead of screenshots.

Tech stack and repo layout

Keep `core/` free of any desktop dependency, so the harness submission packages the core alone and the demo layer can never break the score.

Layer	Choice	Why
Runtime	Python 3.11, asyncio, Pydantic	Matches the kit; cancellation and typed events are native
Local bridge	WebSocket server on localhost	One pipe for both extensions and the overlay
Browser perception	Chrome extension, Manifest V3, TypeScript	DOM text beats OCR on speed and accuracy
Code perception	VS Code extension, TypeScript	Hovers, diagnostics, saves and terminal output come from the editor API
Native perception	pywinauto with the UIA backend	Excel and native apps; you already use it at work
Overlay	Electron: transparent, always on top, click-through	Cursor bubble plus a React view of the multiverse tree; PySide6 if you want one language
Local model	Gemma 4 E4B, 4-bit, via llama.cpp or Ollama	Check E4B support in the version you install

Layer	Choice	Why
Speech	silero-vad, faster-whisper, a fast local TTS	Streaming partials and fast barge-in
Screen capture	mss or dxcam	Only for the vision fallback
Tests	pytest with pytest-asyncio, plus scenario replays	Interruption behaviour is asserted, not just plumbing

```

trail/
  core/
    # harness-scored; no desktop imports
    agent.py      # ParticipantAgent adapter: kit queues <-> bus
    bus.py        # event envelope, pub/sub
    state.py      # SessionState, versions, single writer
    runtime.py    # turn tasks, cancellation, checkpoints
    goals.py      # goal stack, slot dependencies
    classify.py    # interrupt classifier: rules + model hook
    forks.py      # speculative forks and their budgets
    saga.py       # tool tags, rollback, commit barrier, call_id
    arbiter.py    # tiers, flow state, staleness, budget
    trail.py      # session trail store and referent resolver
    llm/          # cloud and local model clients
  desktop/
    bridge.py     # localhost WebSocket hub
    uia.py        # Windows UI Automation reader
    speech.py     # VAD, STT, TTS
    vision.py     # crop plus E4B fallback
  extensions/
    chrome/       # content script, service worker
    vscode/       # hovers, diagnostics, saves, terminal
  overlay/       # Electron: cursor bubble, multiverse tree
  scenarios/     # your own extra interruption scenarios
  tests/

```

Privacy and safety

The first judge question about an always-on screen watcher will be privacy. The answer: it is off by default, visibly on when on, local-first, and it keeps nothing past the session.

- **Opt-in and visible.** Agent mode is off by default and toggled by a hotkey. A coloured ring around the cursor shows whenever perception is running.
- **One-key pause, plus auto-pause.** Perception pauses instantly on the hotkey, and automatically on password managers, banking domains and incognito windows.

- **Redaction at the source.** Password fields, card-like numbers (Luhn check) and one-time codes are masked before they reach the bus.
- **Local-first.** Text extraction runs on the machine. Only the snippet a question needs goes to the cloud model, and screenshots never leave the laptop.
- **Session-only memory.** The trail lives in RAM and is cleared when the session ends. Nothing is written to disk.
- **Keyboard minimalism.** Outside VS Code, only typing rhythm is observed, never key contents.
- **No self-authorized actions.** Irreversible tools wait at the commit barrier for explicit confirmation. The agent cannot press Pay.
- **Page text is data, not instructions.** Hovered web text can contain prompt injections. Trail text is marked untrusted, and tool calls fire only from the user's own intent.
- **Session audit view.** The overlay can list what was read and what was sent to the cloud during this session.

Evaluation

Show three kinds of evidence: the official harness score, interruption metrics from your own scenario suite, and an ablation against a naive cancel-and-restart agent.

Metric	Definition	Target
Harness score	<code>eval_submission.py</code> on the public set, at least 3 reps	Clearly above the ~52/100 reference agent
Time to yield	User activity to the last token or audio frame emitted	Under 150 ms
Correction latency	Final correction to the first token of the revised answer	Under 300 ms on a fork hit
Fork hit rate	Corrections served from a fork, over all corrections	Report it; it is the pitch number
Backchannel false stops	Backchannels that stopped the output	Zero
Stale-output leaks	Outputs delivered with an outdated state version	Zero, asserted in tests
Wasted agent interrupts	Warnings delivered after the user already fixed the issue	Zero
Duplicate writes	Tool writes executed twice	Zero

Your own scenario suite

Write 15–20 scenarios beyond the public set, in the kit's scenario format if it allows custom files. Cover:

- Each of the 7 interrupt types at least twice, including backchannels that must not stop output.
- A correction to a slot with dependent plan steps, so only those steps re-run.
- An interrupt during a reversible tool call, and one during an irreversible call held at the commit barrier.
- A heckler run: 5 interrupts in 30 seconds.
- A topic switch followed by "back to the flight", with constraints intact.

Ablation

Run the same suite with components switched on one at a time:

1. Naive agent: cancel and restart on any user input.
2. Plus checkpoints and the goal stack.
3. Plus the interrupt classifier with duck-then-decide.

4. Plus speculative forks.
5. Plus the saga tool layer.

Plot correction latency and harness score for each variant. A curve that improves at every step is the clearest story you can put in front of judges.

Demo script

The demo runs about 4.5 minutes in three acts and a finale. Every act shows a different interrupt type, with the multiverse tree and a latency counter visible throughout.

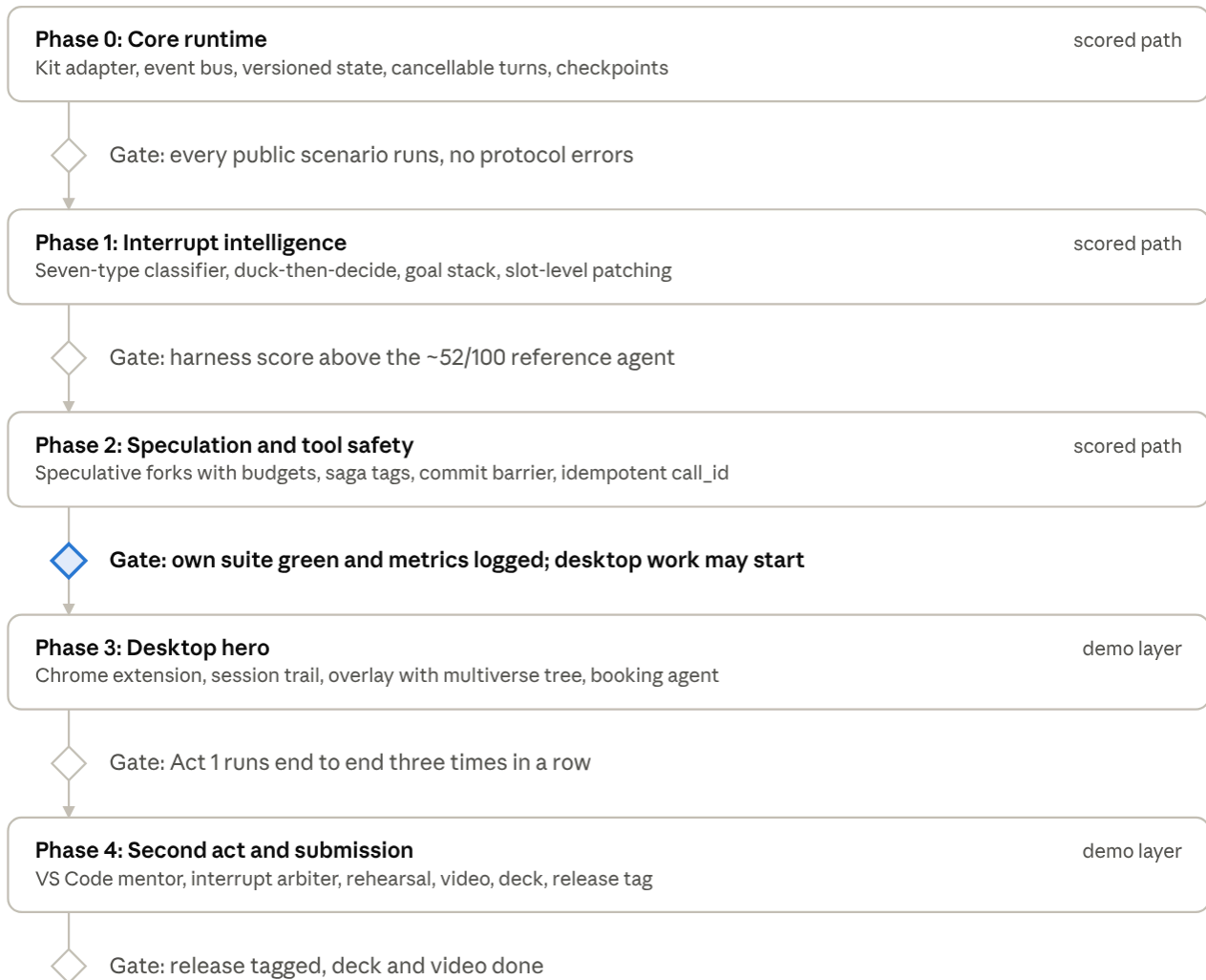
1. **Hook (0:00–0:20).** "Most assistants make you wait your turn. Trail expects to be interrupted." Turn on agent mode; the ring appears around the cursor.
2. **Act 1, booking (0:20–1:40).** On a Chandigarh to Goa results page:
 - a. Hover Friday, Saturday and Sunday, then ask "which should I book?" It answers Saturday, ₹5,000.
 - b. While it is still talking, hover Monday at ₹4,500. It cuts itself off and switches its answer.
 - c. Say "actually, two passengers". One branch of the tree lights up green and the answer lands at once.
 - d. Ask "what's the baggage allowance?", then "back to the flight". The goal is parked and restored with its constraints.
 - e. Say "book it". It holds at the commit barrier and asks for confirmation before paying.
3. **Act 2, cross-app (1:40–2:20).** Switch to the budget sheet and ask "can I afford the cheapest one?". It joins the fare from the browser with the budget cell.
4. **Act 3, code mentor (2:20–3:40).** In VS Code, say "I'm building OAuth login".
 - a. Type a bug. The overlay shows a waiting badge, and the warning arrives when you pause.
 - b. Type a second bug and fix it before pausing. The audit panel shows it dropped as fixed by the user.
 - c. Paste an API key. It interrupts immediately.
 - d. Run the tests, then ask "why did that break?". The diagnosis is already there.
5. **Finale, heckler round (3:40–4:30).** A judge interrupts 5 times in 30 seconds. Trail runs side by side with the naive baseline, with correction latency shown for each. Close on the ablation chart.

Have a scripted replay mode and a recorded video of each act ready, in case the live demo fails on stage.

Build plan

Spend at least the first half of your time on phases 0–2, the scored path. Working solo, desktop work waits for gate 3. With a teammate, they can start phase 3 after gate 1 while the core owner stays on the scored path.

Desktop work starts only once the scored path passes its gates



Build plan · 5 phases, 5 gates

Each gate is a hard check, not a milestone date: work behind a failed gate stops until it passes.

Checklist

Phase 0: Core runtime

- ☐ Read the kit's protocol reference and map every message type to a bus event
- ☐ `ParticipantAgent` adapter over the kit queues
- ☐ `SessionState` with versioning and a single writer

- ☐ Turn as a cancellable task, with a checkpoint on cancel
- ☐ Run every public scenario and fix all protocol errors

Phase 1: Interrupt intelligence

- ☐ Rule classifier for the 7 interrupt types, with duck-then-decide
- ☐ Goal stack with park, resume and abandon
- ☐ Slot dependencies, so a correction re-runs only the affected steps
- ☐ Fillers for slow work
- ☐ Beat the reference agent over 3 reps

Phase 2: Speculation and tool safety

- ☐ Fork manager with budgets and kill-on-version-change
- ☐ Saga tags from the tool manifest, with rollback and compensation
- ☐ Commit barrier and `call_id` idempotency
- ☐ Write 15–20 own scenarios and log the metrics table

Phase 3: Desktop hero

- ☐ Localhost WebSocket bridge
- ☐ Chrome extension: element under pointer, price and date pairing, dwell
- ☐ Trail store and referent resolver
- ☐ Overlay: cursor bubble, perception ring, multiverse tree
- ☐ Booking agent on the kit's travel corpus
- ☐ Voice: VAD barge-in and streaming STT

Phase 4: Second act and submission

- ☐ VS Code extension: hovers, diagnostics, saves, terminal output
- ☐ Interrupt arbiter: tiers, flow state, staleness check, budget
- ☐ Ablation runs and chart
- ☐ Rehearse the demo and record backup videos
- ☐ Deck, 5-minute video and release tag

What the judges weigh

A Theme 02 team's [README](#) lists these weights from the hackathon guidelines. Confirm them for Theme 05.

Criterion	Weight	Where Trail earns it
Working prototype and functionality	30%	Harness score, clean runs, the live demo
Technical depth and feasibility	25%	Runtime mechanisms, tests, latency metrics
Innovation and originality	20%	Pointer interrupts, agent-initiated interrupts, speculative forks
Relevance to theme	15%	Every demo beat is an interruption
Presentation and documentation	10%	Deck, video, this doc, the ablation chart

Submission deliverables

The same README describes these. Check each against your kit.

- Release tag `PRISM_GENAI_HACKATHON_Y2026` on the final commit.
- A slide deck named `CollegeName_TeamName_Submission.pptx`.
- A 5-minute video walkthrough.

Risks and judge Q&A

The biggest risk is building the desktop layer on a core the harness scores poorly. The gates in the build plan exist to prevent exactly that.

Risk	Mitigation
The kit's protocol differs from what public repos describe	Read the protocol reference first; the adapter isolates the core from message formats
The evaluator cannot run a local model	The core runs on the cloud model alone; the local model only speeds up desktop mode
Scope creep across desktop layers	Gates stop work; the PDF, sheet and email agents stay pitch-only unless time remains
Speculation adds cost or latency	Forks run in idle time only, with budgets; switch them off if the hit rate is low
A correction is misread as a backchannel	Output is already ducked; when confidence is low, ask a one-line clarification instead of resuming

Risk	Mitigation
Real booking sites change their markup	Demo on a local results page for a fictional airline, fed by the kit's travel corpus
8 GB of VRAM runs out	4-bit E4B, STT on the CPU, unused apps closed; measure before the demo
The live demo fails on stage	Scripted replay mode, backup videos, a wired mic
Similarity to public entries	Write your own code; the positioning table is your originality case

Judge Q&A

- **"Isn't this Copilot or Gemini screen sharing?"** Those answer questions about the current screen. Trail remembers what you attended to across apps and treats pointer moves as interruptions. The code mentor reviews your work; it never autocompletes.
- **"What if I interrupt during a payment?"** Payment is irreversible, so it waits at the commit barrier. An interrupt before confirmation cancels cleanly and releases the hold.
- **"Won't an agent that interrupts me be annoying?"** Only Critical findings interrupt at once. Everything else waits for a pause or a boundary, is re-checked for staleness, and counts against a budget.
- **"Doesn't speculation burn tokens?"** At most 3 forks, in idle time only, each with a budget, killed on mismatch. The hit rate is reported, so the trade-off is visible.
- **"What do you store?"** Nothing past the session. The trail lives in RAM and is cleared on exit.
- **"Why not a bigger local model?"** The 12B is sized for 16 GB of VRAM. On an 8 GB laptop, E4B handles reflexes and the cloud handles reasoning.

Sources

- [interject](#): a public Theme 05 entry that quotes the brief
- [HarshRaj1607/interruptable-agent](#): a Theme 05 team's notes on the kit layout
- [Samsung-Prism-QUARK](#): a Theme 02 README listing rubric weights and submission deliverables
- [Gemma 4 12B: The Developer Guide](#): 12B sizing and audio support
- [Bringing AI Closer to the Edge and On-Device with Gemma 4](#): E4B and E2B parameters and modalities